

2026 자율연구 | LiFSI / Pyr13FSI 분자동역학

CILE MD 팀원 환경 설정 및 입력 검증 매뉴얼

macOS와 Windows WSL2에서 같은 연구 도구를 설치하고,
공식 5개 조성의 입력이 기술적으로 읽히는지 확인하는 팀 배포용 안내서

기준 버전	GROMACS 2026.3 Packmol 21.2.3 Python 3.13
전달 패키지	CILE_MD_TEAM_PACKAGE_20260901.zip
ZIP SHA-256	2c7fcf0b8a72e47f...a069f8beb23
작성일	2026-09-01

DRAFT INPUTS / NOT FOR PRODUCTION

설치나 20-step smoke 성공은 물리적 평형과 최종 물성을 증명하지 않습니다.

먼저: ZIP과 PDF가 무엇인가

ZIP은 여러 폴더와 파일을 하나로 압축한 전달용 파일이다. 이 PDF는 그 ZIP을 받은 팀원이 운영체제별 설치와 기술 검증을 같은 순서로 수행하도록 만든 전용 매뉴얼이다.

팀원 전달 ZIP 설치 문서, 환경 명세, 공식 5개 조성 입력, 검증 스크립트, fftool과 VMD 예시를 포함한다. 압축 크기는 약 23 MB이다.	기존 교수 전달 ZIP CILE_MD_Handoff_v0_DRAFT.zip은 입력과 기술 검증만 담은 별도의 작은 초안이다. 팀원 ZIP은 여기에 환경 설정 자료를 더한 것이다.
--	--

포함 범위

포함	제외
Mac / Windows 설치 안내	전체 1.1 GB 연구 작업 폴더
Brewfile, environment.yml	장시간 trajectory, checkpoint, energy
5개 조성 좌표, topology, MDP	개인 Obsidian, Drive 정보, 로컬 백업
validate_inputs, validate_grompp, smoke	보고서 작업본과 개인 메모
fftool, VMD 예시 구조	물리적 평형이나 production 보증

가장 중요한 경계

이 패키지는 환경 설정과 기술 입력 검증용이다. 다른 컴퓨터에서 설치와 smoke가 끝나기 전에는 재현되었다고 말할 수 없으며, smoke 성공도 과학적 결과 검증이 아니다.

문서 구성

구역	내용
빠른 시작	운영체제별 전체 흐름과 공통 기준
macOS	Homebrew, Conda, VMD, fftool
Windows	WSL2, Ubuntu, Miniforge, Windows GUI 도구
공통 검증	ZIP 체크섬, 정적 검사, grompp, smoke
문제 해결	자주 만나는 오류와 팀 보고 양식

02 10분 안에 전체 경로 파악

빠른 시작

아래 단계는 설치 순서를 요약한 지도다. 실제 명령은 각 운영체제 장을 따른다.

macOS 경로

단계	할 일	완료 근거
1	ZIP 압축 해제	README와 두 설치 문서가 보임
2	Xcode Tools + Homebrew	git, brew 명령이 인식됨
3	brew bundle	gmx, packmol, pandoc 경로 확인
4	Conda 환경 생성	Python 3.13과 핵심 import 성공
5	VMD / fftool	VMD 구조 표시, fftool -h 출력
6	공통 검증	checksum -> static -> grompp 순서 통과

Windows 경로

단계	할 일	완료 근거
1	PowerShell에서 WSL2 + Ubuntu	wsl --list --verbose에 VERSION 2
2	Ubuntu 기본 도구	git, curl, build-essential 설치
3	WSL 내부 Miniforge	Ubuntu에서 conda 명령 인식
4	프로젝트를 WSL 홈으로 복사	/home/사용자/cile-md에서 파일 확인
5	GROMACS / Packmol / Python	기준 버전과 경로 확인
6	Windows VMD + 공통 검증	GUI 표시 및 WSL 검사 통과

명령 입력 위치를 섞지 않기

Windows에서는 PowerShell과 Ubuntu/WSL이 서로 다른 환경이다. WSL 설치 명령은 PowerShell, GROMACS와 Conda 명령은 Ubuntu에서 실행한다.

03 설치 전 공통 점검

컴퓨터와 파일을 먼저 확인하기

환경 차이는 계산 실패뿐 아니라 서로 다른 결과 해석으로 이어질 수 있다. 설치 전에 운영체제, CPU 아키텍처, 저장 공간과 프로젝트 위치를 기록한다.

macOS 정보 수집

```
uname -m
sw_vers
system_profiler SPHardwareDataType | sed -n '1,20p'
df -h .
```

Windows PowerShell 정보 수집

```
Get-ComputerInfo | Select-Object `
    WindowsProductName, WindowsVersion, OsArchitecture, CsTotalPhysicalMemory
Get-PSDrive C
```

권장 폴더 위치

macOS

~/Research/cile-md
경로를 단순하게 유지한다. 공백과 한글은 인용부호를 쓰면 되지만 팀원별 경로가 다르면 기록한다.

Windows WSL2

/home/■■■■/cile-md
/mnt/c에서 장시간 계산하지 않는다. Linux 홈이 파일 I/O와 권한 처리에 유리하다.

팀원이 시작 전에 체크할 항목

체크	확인 내용
☐	전달받은 ZIP 파일명이 정확하다
☐	ZIP 옆의 .sha256 파일도 함께 받았다
☐	운영체제와 CPU가 무엇인지 기록했다
☐	프로젝트와 생성 파일을 저장할 여유 공간이 있다
☐	설치 권한과 인터넷 연결을 확인했다
☐	장시간 계산은 아직 시작하지 않는다

현재 Git 상태

원본 프로젝트는 아직 첫 커밋이 없는 로컬 저장소다. 현재는 git clone만으로 전체 연구 폴더를 받을 수 없으므로 이 ZIP을 사용한다.

04 macOS 설치

macOS 핵심 계산 환경

기본 Mac은 Apple Silicon이다. Intel Mac도 가능하지만 Homebrew, Conda, VMD 설치 파일은 아키텍처에 맞춰야 한다.

1. Xcode Command Line Tools

```
xcode-select --install
git --version
```

2. Homebrew

공식 사이트 <https://brew.sh/>의 설치 명령을 사용한다. 설치 후 새 터미널에서 확인한다.

```
brew --version
```

3. GROMACS, Packmol, Pandoc

```
cd "$HOME/Research/cile-md"
brew bundle --file Brewfile

command -v gmx
gmx --version
command -v packmol
brew list --versions gromacs packmol pandoc
```

버전 차이가 보이면

기본은 GROMACS 2026.3과 Packmol 21.2.3이다. Homebrew가 더 새 버전을 설치했다면 팀 기록에 남기고, 바로 장시간 계산을 시작하지 않는다.

4. Miniforge / Conda 환경

새 팀원에게는 Miniforge를 권장한다. Apple Silicon은 macOS arm64, Intel은 macOS x86_64 설치 파일을 선택한다.

```
cd "$HOME/Research/cile-md"
conda env create -f environment.yml
conda activate cile-md-report

python --version
python -c "import numpy, pandas, scipy, matplotlib, seaborn, yaml; print('PYTHON_PACKAGES_OK')"
```

같은 이름의 Conda 환경이 이미 있는 경우

환경을 덮어쓰지 않는다. `conda env list` 결과를 팀에 공유하고 기존 환경을 유지할지 새 이름으로 만들지 결정한다.

05 macOS 도구와 확인

fftool, VMD와 선택 도구

fftool

fftool은 ZIP 안에 포함되어 있다. 별도 PATH 설치보다 활성화한 Conda의 Python으로 직접 확인한다.

```
cd "$HOME/Research/cile-md"
conda activate cile-md-report
python 90_Reproduce_Scripts/tools/fftool/fftool -h
```

VMD

VMD 공식 사이트에서 Mac 아키텍처에 맞는 빌드를 설치하고 Applications 폴더에 둔다.

```
00_START_HERE/OPEN_VMD_50ps_animation.command
00_START_HERE/OPEN_VMD_P2_series.command
```

실행 파일은 /Applications와 ~/Applications의 VMD 앱을 찾는다.

선택 도구

```
brew install ffmpeg pymol

ffmpeg -version
pymol --version
latexmk --version
pandoc --version
```

macOS 설치 완료 판정

항목	통과 기준
GROMACS	gmx --version에서 버전과 실행 경로 확인
Packmol	command -v packmol이 경로 반환
Python	3.13.x 및 핵심 패키지 import 성공
fftool	도움말 45줄 전후 출력, 오류 없음
VMD	101 MODEL / 101 ENDMDL 구조가 정상 표시

VMD에서 구조가 보인다는 의미

PDB 프레임을 읽고 표시할 수 있다는 뜻이다. 50 ps 예시가 보인다고 해서 그 시스템이 평형에 도달했거나 diffusion, conductivity가 유효하다는 뜻은 아니다.

Windows는 WSL2 + Ubuntu로 계산

Windows 네이티브 GROMACS 직접 빌드는 MSVC, CMake, FFT 설정 차이가 생기기 쉽다. 팀의 공통 계산 경로는 WSL2 Ubuntu로 고정한다.

1. 관리자 PowerShell

```
wsl --install -d Ubuntu
```

재부팅 후 Ubuntu를 한 번 실행해 Linux 사용자 이름과 비밀번호를 만든다. PowerShell에서 버전을 확인한다.

```
wsl --list --verbose
```

통과 기준

Ubuntu 행의 VERSION이 2여야 한다. 공식 안내: [Microsoft WSL 설치 문서](#)

2. Ubuntu / WSL 터미널

```
sudo apt update
sudo apt install -y git curl wget build-essential ca-certificates

uname -m
git --version
```

3. WSL 내부 Miniforge

WSL 아키텍처에 맞는 Linux 설치 파일을 사용한다.

PC 종류	uname -m	Miniforge 설치 파일
일반 Intel / AMD	x86_64	Miniforge3-Linux-x86_64.sh
Windows ARM	aarch64	Miniforge3-Linux-aarch64.sh

Windows Anaconda와 WSL Conda는 별개

Windows에 Anaconda가 있어도 WSL Ubuntu에서는 보통 그 환경을 사용하지 않는다. 계산용 Miniforge는 WSL 내부에 설치한다.

07 Windows 도구와 확인

WSL 계산 환경과 Windows GUI

1. 프로젝트를 WSL 홈으로 복사

```
# Recommended WSL path
/home/<linux-user>/cile-md

# Open from Windows Explorer
\\wsl$\Ubuntu\home\<linux-user>
```

2. Python 환경

```
cd "$HOME/cile-md"
conda env create -f environment.yml
conda activate cile-md-report
```

3. GROMACS와 Packmol

```
conda install -c conda-forge gromacs=2026.3
python -m pip install packmol==21.2.3

command -v gmx
gmx --version
command -v packmol
python --version
```

4. fftool

```
python 90_Reproduce_Scripts/tools/fftool/fftool -h
```

5. Windows에 직접 설치할 GUI

프로그램	설치 위치	용도
VMD	Windows	PDB / trajectory 시각화
PyMOL	Windows	추가 구조 표시와 PNG frame
FFmpeg	Windows	frame을 MP4로 변환
Windows Terminal	Windows	PowerShell과 WSL 관리
GROMACS / Packmol	WSL Ubuntu	계산과 입력 생성

Windows VMD에서 직접 열 파일

```
03_Analysis_Results/VMD_View/01_all_five_50ps_animation.pdb
03_Analysis_Results/VMD_View/VMD_01_open_50ps_animation.tcl
```

macOS .command 파일은 Windows에서 실행하지 않음

Windows에서는 PDB를 VMD로 직접 열고 필요하면 Tk Console에서 Tcl 파일을 불러온다.

08 공통 기술 검증

항상 같은 순서로 확인하기

검증은 아래 네 층으로 분리한다. 앞 단계가 통과하지 않았으면 다음 단계로 넘어가지 않는다.

순서	검증	실제 수행	증명하지 않는 것
1	ZIP / SHA	전달 파일 무결성	GROMACS 실행
2	validate_inputs	파일, 원자수, topology 정적 검사	TPR 생성, MD
3	validate_grompp	5개 조성 EM-stage TPR 생성	trajectory, 안정성
4	20-step smoke	EM->NVT->NPT->prod 연결	평형, production 타당성

1. 패키지 내부 체크섬

```
# macOS
shasum -a 256 -c SHA256SUMS

# Windows WSL2 / Ubuntu
sha256sum -c SHA256SUMS
```

2. 정적 입력 검사

```
cd 07_Handoff/CILE_MD_Handoff_v0_DRAFT
bash validate_inputs.sh
```

성공 문구: Static validation PASSED

3. GROMACS 전처리

```
bash validate_grompp.sh
```

성공 문구: grompp validation PASSED for all five systems.

4. 기술 smoke - 담당자 확인 후

```
THREADS=2 bash run_smoke_all.sh
```

20-step smoke의 의미

각 단계가 연결되고 출력 파일이 생성되지만 본다. 짧게 제한한 EM은 충분히 수렴하지 않을 수 있고, 밀도와 box 안정성 및 열역학적 평형은 별도 장시간 분석이 필요하다.

어떤 파일을 어디에서 찾는가

```
CILE_MD_TEAM_PACKAGE_20260901/
■■■ README_FIRST_KOR.md
■■■ Brewfile
■■■ environment.yml
■■■ MANIFEST.tsv
■■■ SHA256SUMS
■■■ 00_START_HERE/
■■■ 03_Analysis_Results/VMD_View/
■■■ 07_Handoff/CILE_MD_Handoff_v0_DRAFT/
■■■ 08_Next_Research/03_Environments/
■■■ 90_Reproduce_Scripts/tools/fftool/
```

핵심 파일 지도

파일 / 폴더	용도
README_FIRST_KOR.md	패키지 범위와 가장 먼저 할 일
TEAM_SETUP_*	운영체제별 상세 설치 명령
Brewfile	Mac 네이티브 과학 도구
environment.yml	Python 3.13 분석 패키지
systems/*	L1P1, L1P2, L1P3, L2P1, L3P1 입력
mdp/*	EM, NVT, NPT, production 설정
validation/*	과거 로컬 기술 검증 증거와 매트릭스
MANIFEST.tsv	파일 경로, 바이트 수, 해시 목록
SHA256SUMS	압축 해제 후 파일 무결성 검사

현재 제작본 검증 기록

항목	관찰 결과
ZIP 압축 검사	No errors detected
내부 SHA-256	108개 파일 OK
정적 입력 검사	PASS
grompp	5개 조성 5/5 PASS
fftool	도움말 실행 확인
VMD PDB 프레임	두 파일 모두 MODEL 101 / ENDMDL 101

자주 만나는 오류

증상	가능한 원인	먼저 할 일
gmx: command not found	Homebrew / Conda 비활성	command -v gmx, conda activate 확인
env: python: No such file	fftool shebang이 Python을 못 찾음	Conda 활성화 후 python fftool -h 사용
Packmol을 못 찾음	PATH 또는 설치 누락	command -v packmol, 버전 기록
VMD 앱을 못 찾음	잘못된 아키텍처 / 설치 위치	Applications 설치, VMD*.app 이름 확인
WSL VERSION 1	WSL2 전환 안 됨	PowerShell에서 WSL 버전 확인 및 전환
/mnt/c에서 매우 느림	Windows 파일시스템 경유	프로젝트를 WSL 홈으로 복사
grompp 실패	버전 / topology / 경로 문제	로그 저장, -maxwarn 추가 금지
checksum FAILED	전송 손상 / 파일 변경	계산 중지 후 ZIP 다시 받기

하지 말아야 할 우회

- -maxwarn을 임의로 추가해 grompp 경고를 넘기지 않는다.
- 동작하는 팀원의 전체 Conda base 환경을 그대로 복사하지 않는다.
- 50 ps 예시 구조나 20-step smoke를 최종 연구 결과로 사용하지 않는다.
- Windows와 WSL 경로를 한 명령에서 섞지 않는다.
- 버전 차이를 기록하지 않은 채 장시간 계산을 시작하지 않는다.

오류 보고에 반드시 포함할 것

```
OS / CPU:
Terminal (PowerShell, WSL, or Mac Terminal):
Exact command:
Full error message:
gmx --version:
python --version:
command -v gmx packmol python:
Failed stage (checksum / static / grompp / smoke):
```

오류가 나면 먼저 기록

같은 오류가 여러 컴퓨터에서 반복되는지 확인한 뒤 스크립트나 환경 명세를 한꺼번에 수정한다. 한 컴퓨터에서만 임시 우회하면 팀 재현성이 더 나빠진다.

설치 완료 보고서

Mac 팀원이 제출할 출력

```
uname -m
sw_vers -productVersion
gmx --version | sed -n '1,20p'
brew list --versions packmol
python --version
conda env list
command -v gmx packmol python
```

Windows 팀원이 제출할 출력

```
# Ubuntu / WSL
uname -a
cat /etc/os-release
gmx --version | sed -n '1,20p'
python --version
conda env list
command -v gmx packmol python

# PowerShell
wsl --list --verbose
```

검증 수준을 정확히 쓰기

수준	의미	현재 상태
Implemented	파일, 명령, 설정이 존재	팀원 ZIP과 PDF 존재
Unit-verified	정적 / 전처리 검사가 통과	현재 Mac에서 SHA, static, grompp 통과
Simulator-verified	시뮬레이터 사용자 경로	해당 없음
Physical-device-verified	실제 장비에서 사용자 경로	기존 개인 Mac 기술 경로만 부분 확인
Not verified / 미검증	아직 관찰하지 않은 경로	팀원 Mac / Windows WSL2, full simulation

연구적으로 아직 증명되지 않은 것

목표 밀도, box 안정성, 충분한 EM 수렴, 온도 / 압력 / 에너지 plateau, 독립 replica 일치, diffusion과 conductivity 타당성은 full-length 실행과 QC가 필요하다. 기술 설치 성공으로 대체할 수 없다.

최종 팀원 체크리스트

아래 항목을 한 장비에서 모두 확인한 뒤 설치 결과를 팀에 제출한다. 다른 장비의 성공 기록을 자신의 검증으로 대신하지 않는다.

체크	완료 조건
☐	ZIP SHA-256와 내부 108개 체크섬 통과
☐	기준 버전과 실제 버전 기록
☐	validate_inputs 통과
☐	validate_grompp 5/5 통과
☐	fftool 도움말 실행 확인
☐	VMD에서 예시 구조 표시 확인
☐	승인 후에만 20-step smoke 실행
☐	결과를 팀 양식으로 제출

팀에 제출할 것

운영체제와 CPU, 도구 버전, 명령 경로, checksum / static / grompp 결과, 실패가 있다면 전체 로그를 제출한다.

아직 하지 않을 것

full-length MD, 새 물성 계산, -maxwarn 우회, GPU 설정 변경은 프로토콜과 담당자 승인을 받은 뒤 진행한다.

완료 보고 문장 예시

내 장비에서 ZIP 무결성, 정적 입력 검사와 5개 조성 grompp를 확인했다. smoke와 full-length 시뮬레이션은 별도 상태로 보고한다.

공식 설치 자료

도구	공식 주소
Homebrew	https://brew.sh/
Miniforge	https://github.com/conda-forge/miniforge
Microsoft WSL	https://learn.microsoft.com/windows/wsl/install
GROMACS	https://manual.gromacs.org/documentation/current/install-guide/
Packmol	https://github.com/m3g/packmol
VMD	https://www.ks.uiuc.edu/Research/vmd/
PyMOL	https://pymol.org/

권장 전달 방식

팀원에게 CILE_MD_TEAM_PACKAGE_20260901.zip, ZIP의 .sha256, 그리고 이 PDF 세 파일을 함께 전달한다. 팀원은 PDF를 먼저 읽고 ZIP을 풀어 운영체제별 안내서를 따른다.

핵심 원칙

같은 프로그램을 설치했다는 사실보다, 같은 입력과 명령이 실제로 읽히고 동일한 검증 단계가 통과했는지를 기록하는 것이 중요하다.

설치 -> 정적 검사 -> grompp -> 승인된 smoke -> 장시간 연구 검증